

Instruction Set Architecture II

Michael C. Hackett

Associate Professor, Computer Science



Lecture Topics

- Floating Point Instructions
- Floating Point Data Representation
- Floating Point Instruction Formats
 - Floating Point Register Format
 - Floating Point Immediate Format

Floating Point Instructions

- For non-integer numbers, the **floating-point** data type is used.
 - Examples of non-integers: 5.678 and -22.1
- MIPS has two floating point data types- *float* and *double*
 - These are identical to the float and double types in Java
- The float type is a 32-bit (single precision) number
- The double type is a 64-bit (double precision) number
 - We will discuss what *single precision* and *double precision* mean later in this lecture. For now, you only need to remember the size (in bits) of each.

Floating Point Instructions

- MIPS has 32 additional registers that are used for holding floating point numbers.
 - Registers \$f0 through \$f31
- Each of these registers, like the registers previously seen, are 32 bits in size
- The registers are in a separate part of the CPU
 - Coprocessor 1

Floating Point Instructions

Registers	Coproc 1	Coproc 0
Name	Float	Double
\$f0	0x00000000	0x0000000000000000
\$f1	0x00000000	
\$f2	0x00000000	0x0000000000000000
\$f3	0x00000000	
\$f4	0x00000000	0x0000000000000000
\$f5	0x00000000	
\$f6	0x00000000	0x0000000000000000
\$f7	0x00000000	
\$f8	0x00000000	0x0000000000000000
\$f9	0x00000000	
\$f10	0x00000000	0x0000000000000000
\$f11	0x00000000	
\$f12	0x00000000	0x0000000000000000
\$f13	0x00000000	
\$f14	0x00000000	0x0000000000000000
\$f15	0x00000000	
\$f16	0x00000000	0x0000000000000000
\$f17	0x00000000	
\$f18	0x00000000	0x0000000000000000
\$f19	0x00000000	
\$f20	0x00000000	0x0000000000000000
\$f21	0x00000000	
\$f22	0x00000000	0x0000000000000000
\$f23	0x00000000	
\$f24	0x00000000	0x0000000000000000
\$f25	0x00000000	
\$f26	0x00000000	0x0000000000000000
\$f27	0x00000000	
\$f28	0x00000000	0x0000000000000000
\$f29	0x00000000	
\$f30	0x00000000	0x0000000000000000
\$f31	0x00000000	

Floating Point Instructions

- To initialize memory with a single-precision value, the **.float** directive is used in the program's data section.

.data

x: .float 6.789

Floating Point Instructions

- The **l.s** (load single-precision) mnemonic loads a single precision non-integer from memory to a register

l.s \$fd, label

- \$fd = Destination Register
- label = Symbolic (or non-symbolic) memory reference

l.s \$f0, x

Floating Point Instructions

- The **mov.s** (**move** single-precision) mnemonic moves a single precision non-integer from one register to another

mov.s \$fd, \$fs

- \$fd = Destination Register
- \$fs = Source Register

mov.s \$f6, \$f0

Floating Point Instructions

- Arithmetic for single precision floating-point data:

add.s	\$fd, \$fs, \$ft
sub.s	\$fd, \$fs, \$ft
mul.s	\$fd, \$fs, \$ft
div.s	\$fd, \$fs, \$ft

- \$fd = Destination Register
- \$fs = Source Register 1
- \$ft = Source Register 2

Floating Point Instructions

- Special instructions are used to compare the equality of single-precision values.
- The **c.eq.s** (compare **e**quality **s**ingle-precision) mnemonic compares two floats for equality

c.eq.s \$fs, \$ft

- \$fs = Source Register 1
 - \$ft = Source Register 2
-
- A 1 is stored to a special 1-bit condition code if the result is true
 - A 0 is stored to the condition code if the result is false

Floating Point Instructions

- The **c.lt.s** (compare less than single-precision) mnemonic compares two floats for a less than relationship

c.lt.s \$fs, \$ft

- \$fs = Source Register 1
- \$ft = Source Register 2
- A 1 is stored to the condition code if the result is true
- A 0 is stored to the condition code if the result is false

Floating Point Instructions

- The **c.le.s** (compare less than or equal single-precision) mnemonic compares two floats for a less than or equal relationship

c.le.s \$fs, \$ft

- \$fs = Source Register 1
- \$ft = Source Register 2
- A 1 is stored to the condition code if the result is true
- A 0 is stored to the condition code if the result is false

Floating Point Instructions

- To branch after a comparison, the **bc1t** mnemonic branches to a specified label if the condition code contains 1

bc1t label

- label = Section to jump to if the condition code contains 1

Floating Point Instructions

- To branch after a comparison, the **bc1f** mnemonic branches to a specified label if condition code contains 0

bc1f label

- label = Section to jump to if condition code contains 0

Floating Point Instructions

- For 64-bit double-precision values, pairs of registers are used.
 - Though, only the first register is specified in instructions
- By convention, the even numbered registers are used as the first register of a double-precision number.
 - \$f0 and \$f1
 - \$f2 and \$f3
 - \$f4 and \$f5
 - and so on...

Floating Point Instructions

- To initialize memory with a double-precision value, the **.double** directive is used in the program's data section.

.data

x: .float 6.789

y: .double 3.1415

Floating Point Instructions

- The **l.d** (load **d**ouble-precision) mnemonic loads a double-precision non-integer from memory to a register

l.d \$fd, label

- \$fd = Destination Register
- label = Symbolic (or non-symbolic) memory reference

l.d \$f0, y

Floating Point Instructions

- The **mov.d** (**move double-precision**) mnemonic moves a double precision non-integer from one register pair to another

mov.d \$fd, \$fs

- \$fd = Destination Register
- \$fs = Source Register

mov.d \$f6, \$f0

- The value in pair \$f0 and \$f1 will be moved to the register pair \$f6 and \$f7

Floating Point Instructions

- Arithmetic for double precision floating-point data:

add.d	\$fd, \$fs, \$ft
sub.d	\$fd, \$fs, \$ft
mul.d	\$fd, \$fs, \$ft
div.d	\$fd, \$fs, \$ft

- \$fd = Destination Register
- \$fs = Source Register 1
- \$ft = Source Register 2

Floating Point Instructions

- The **c.eq.d** (compare **e**quality **d**ouble-precision) mnemonic compares two doubles for equality

c.eq.d \$fs, \$ft

- \$fs = Source Register 1
 - \$ft = Source Register 2
-
- A 1 is stored to the condition code if the result is true
 - A 0 is stored to the condition code if the result is false

Floating Point Instructions

- The **c.lt.s** (compare less than double-precision) mnemonic compares two doubles for a less than relationship

c.lt.d \$fs, \$ft

- \$fs = Source Register 1
- \$ft = Source Register 2
- A 1 is stored to the condition code if the result is true
- A 0 is stored to the condition code if the result is false

Floating Point Instructions

- The **c.le.d** (compare less than or equal double-precision) mnemonic compares two doubles for a less than or equal relationship

c.le.d \$fs, \$ft

- \$fs = Source Register 1
 - \$ft = Source Register 2
-
- A 1 is stored to the condition code if the result is true
 - A 0 is stored to the condition code if the result is false

Floating Point Instructions

- To branch after a comparison, the process is the same as it was for floats
 - The **bc1t** mnemonic branches to a specified label if the condition code contains 1

bc1t label

- label = Section to jump to if the condition code contains 1
- The **bc1f** mnemonic branches to a specified label if the condition code contains 0

bc1f label

- label = Section to jump to if the condition code contains 0

Floating Point Data Representation

- While 101.11 is a perfectly valid base 2 number, we have no way to specify a “binary point” in memory.
 - Like how we don’t have a way to specify a “-” for a negative number

1 0 1 . 1 1

2^2 2^1 2^0 2^{-1} 2^{-2}

$$(1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) + (1 \times 2^{-2})$$

$$(1 \times 4) + (0 \times 2) + (1 \times 1) + (1 \times 0.5) + (1 \times 0.25)$$

$$4 + 0 + 1 + 0.5 + 0.25 = 5.75_{10}$$

Floating Point Data Representation

- Two such number formats for representing non-integers are:
 - Fixed Point Notation
 - Floating Point Notation
- Floating point is the more common of the two.

Floating Point Data Representation

- Fixed Point Notation

- A specific number of bits allocated for digits to the left of the binary point.
- A specific number of bits allocated for digits to the right of the binary point.

Binary	Decimal
000	0.0
001	0.5
010	1.0
011	1.5
100	2.0
101	2.5
110	3.0
111	3.5

Binary	Decimal
000	0.00
001	0.25
010	0.50
011	0.75
100	1.00
101	1.25
110	1.50
111	1.75

Floating Point Data Representation

- In floating-point, non-integers are represented via scientific notation.
 - The decimal number 12.41 could be re-written as 1.241×10^1
 - The decimal number -0.003 could be rewritten as -3.0×10^{-2}
- The binary number 1001.1 is expressed in scientific notation as $1.0011_2 \times 2^3$
 - Where $2^3 = 4 = 100_2$
 - The number 1.0011 is referred to as the significand or *mantissa*; The digits to the right of the binary point are called the *fraction*.

Floating Point Data Representation

$$1.01101 \times 2^3 = 1011.01$$

$$-1.11 \times 2^{-1} = -0.111$$

- The placement of the binary point “floats” to where it needs to be.

Floating Point Data Representation

- We'll begin with half-precision (16-bits) floating point format
- Memory space is allotted for:
 - A sign bit – 1 bit
 - The exponent – 5 bits
 - The mantissa – 10 bits
- (This example will use 16 bits to store 11.25, or 1.01101×2^3)
- This number is positive, so the sign bit is zero.

Sign Bit	Exponent	Fraction
0		

Floating Point Data Representation

- 1.01101×2^3
- The exponent is expressed as $2^{b-1}-1$ more than the exponent's actual value.
 - Where b is number of bits allotted for the exponent.
 - $3 + (2^{5-1}-1) = 3 + (2^4-1) = 3 + 15 = 18 = 10010_2$

Sign Bit	Exponent	Fraction
0	10010	

Floating Point Data Representation

- 1.01101×2^3
- The mantissa is supposed to start with “1.” so only the fractional bits will be stored.

Sign Bit	Exponent	Fraction
0	10010	0110100000

Floating Point Data Representation

- Working backwards with a different number:

Sign Bit	Exponent	Fraction
1	10011	0110100000

- A 1 in the sign bit means the number is negative.
- Exponent: $10011_2 = 19 \rightarrow 19 - (2^{5-1}-1) = 19 - (2^4-1) = 19 - 15 = 4$
- Mantissa: 1.01101

$$-1.01101_2 \times 2_{10}^4 = -1.01101_2 \times 10000_2 = -10110.1_2 = -22.5_{10}$$

Floating Point Data Representation

- Next, we'll demonstrate the same number but this time use single-precision (32-bits) floating point format
- Memory space is allotted for:
 - A sign bit – 1 bit
 - The exponent – 8 bits
 - The mantissa – 23 bits
- (This example will use 32 bits to store 11.25, or 1.01101×2^3)
 - This number is positive, so the sign bit is zero.

Sign Bit	Exponent	Fraction
0		

Floating Point Data Representation

- 1.01101×2^3
- The exponent is again expressed as $2^{b-1}-1$ more than the exponent's actual value.
 - Where b is number of bits allotted for the exponent.
 - $3 + (2^{8-1}-1) = 3 + (2^7-1) = 3 + 127 = 130 = 10000010_2$

Sign Bit	Exponent	Fraction
0	10000010	

Floating Point Data Representation

- 1.01101×2^3
- The mantissa is supposed to start with “1.” so only the fractional bits will be stored.

Sign Bit	Exponent	Fraction
0	10000010	011010000000000000000000

Floating Point Data Representation

- Finally, we'll demonstrate the same number using double-precision (64-bits) floating point format
- Memory space is allotted for:
 - A sign bit – 1 bit
 - The exponent – 11 bits
 - The mantissa – 52 bits
- (This example will use 64 bits to store 11.25, or 1.01101×2^3)
 - This number is positive, so the sign bit is zero.

Sign Bit	Exponent	Fraction
0		

Floating Point Data Representation

- 1.01101×2^3
- The exponent is again expressed as $2^{b-1}-1$ more than the exponent's actual value.
 - Where b is number of bits allotted for the exponent.
 - $3 + (2^{11-1}-1) = 3 + (2^{10}-1) = 3 + 1023 = 1026 = 10000000010_2$

Sign Bit	Exponent	Fraction
0	10000000010	

Floating Point Data Representation

- 1.01101×2^3
- The mantissa is supposed to start with “1.” so only the fractional bits will be stored.

[illegible]

Floating Point Data Representation

- Half Precision – 16 bits
 - One sign bit
 - Five exponent bits
 - Ten fractional bits
- Single Precision – 32 bits
 - One sign bit
 - Eight exponent bits
 - Twenty-three fractional bits
- Double Precision – 64 bits
 - One sign bit
 - Eleven exponent bits
 - Fifty-two fractional bits

Floating Point Instruction Formats

- MIPS has two floating point instruction formats:

- **Floating Point Register Format (FR Format)**

add.s \$f2, \$f0, \$f1

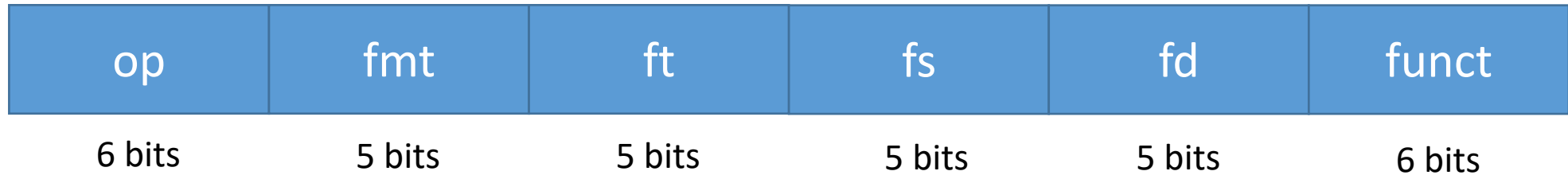
- **Floating Point Immediate Format (FI Format)**

- Only used for conditional branch instructions

bc1t label

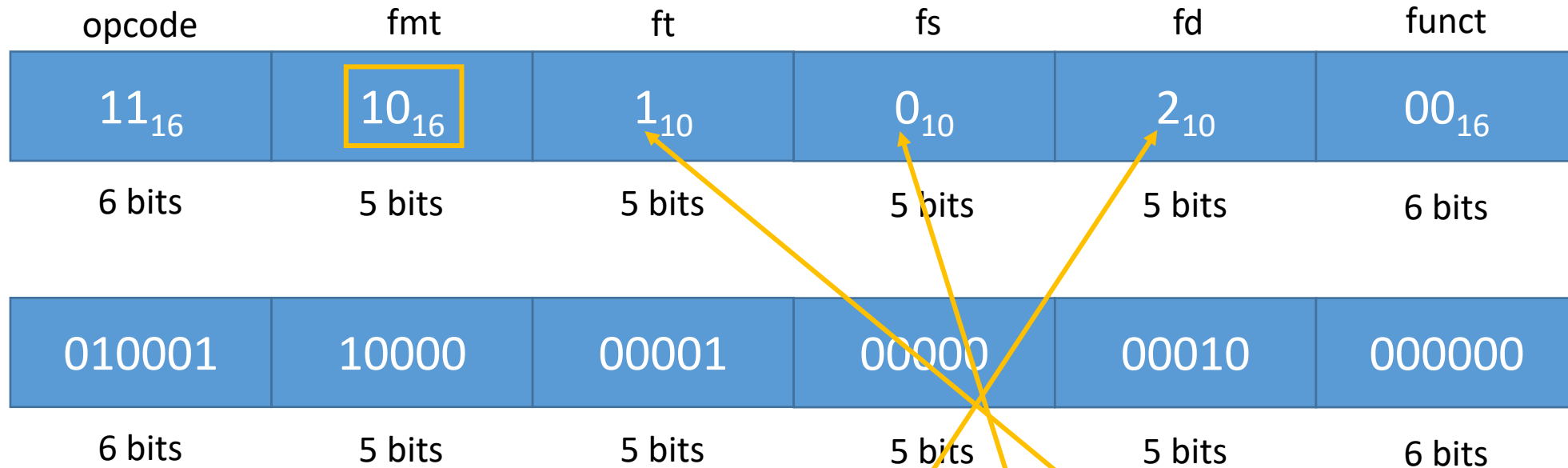
Floating Point Register Format Instructions

- MIPS Floating Point Register Format Instruction



- op – Opcode
- fmt – Format (indicates single or double precision)
- ft – Second source register (right operand)
- fs – First source register (left operand)
- fd – Destination register
- funct – Function code (specifies a variant of the opcode)

Floating Point Register Format Instructions



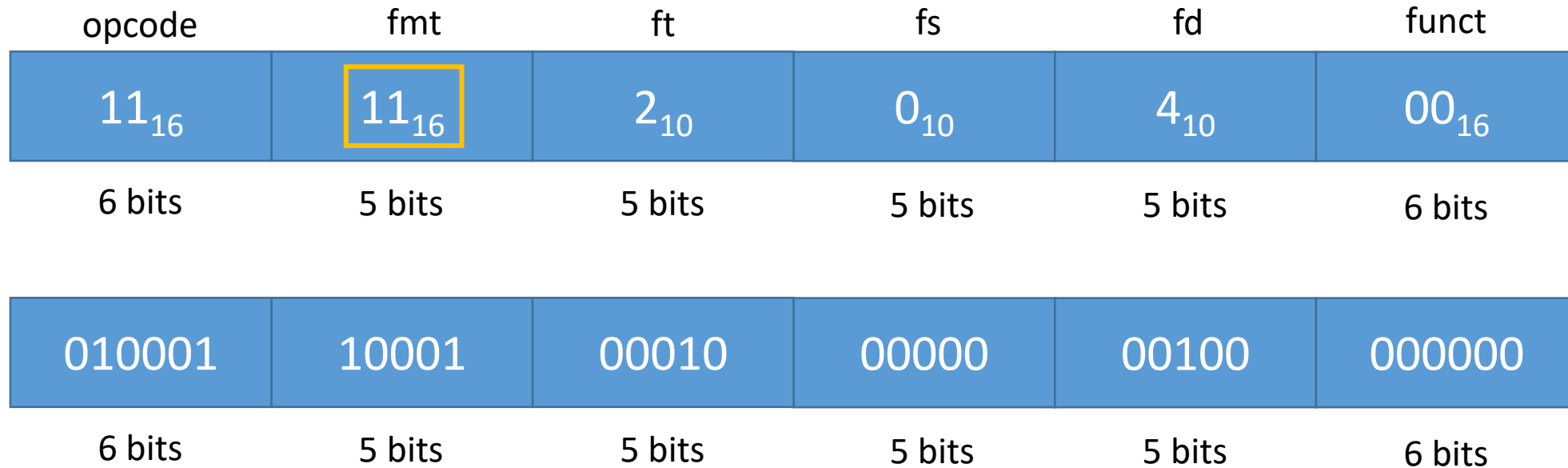
- Assembly Language:

add.s \$f2, \$f0, \$f1

- Machine Language:

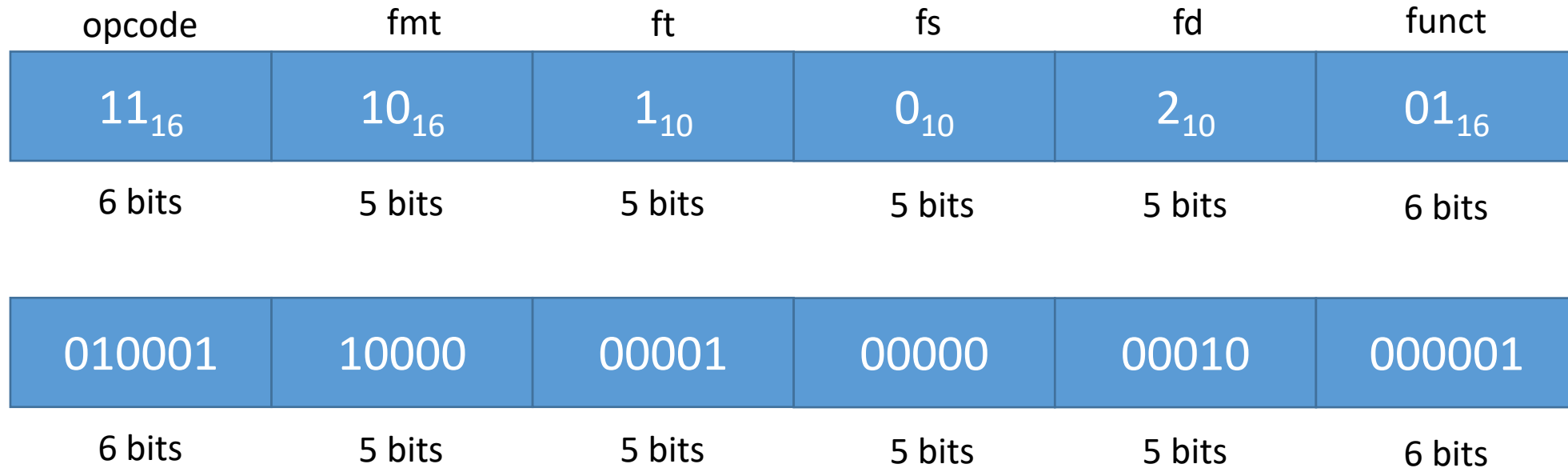
0100 0110 0000 0001 0000 0000 1000 0000

Floating Point Register Format Instructions



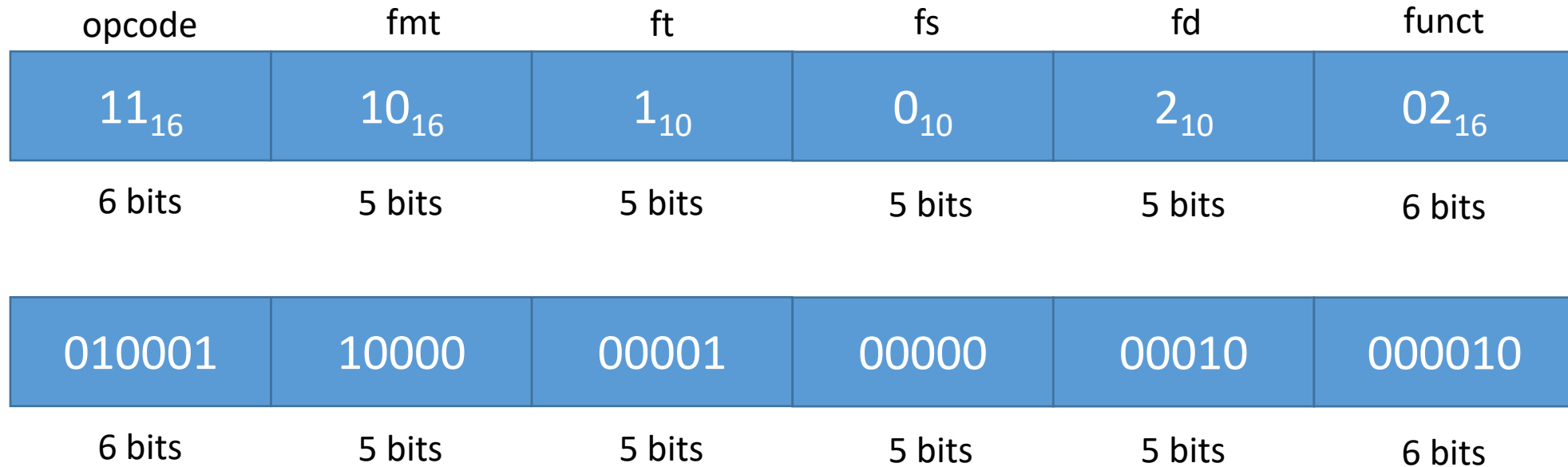
- Assembly Language: `add.d $f4, $f0, $f2`
- Machine Language: `0100 0110 0010 0010 0000 0001 0000 0000`

Floating Point Register Format Instructions



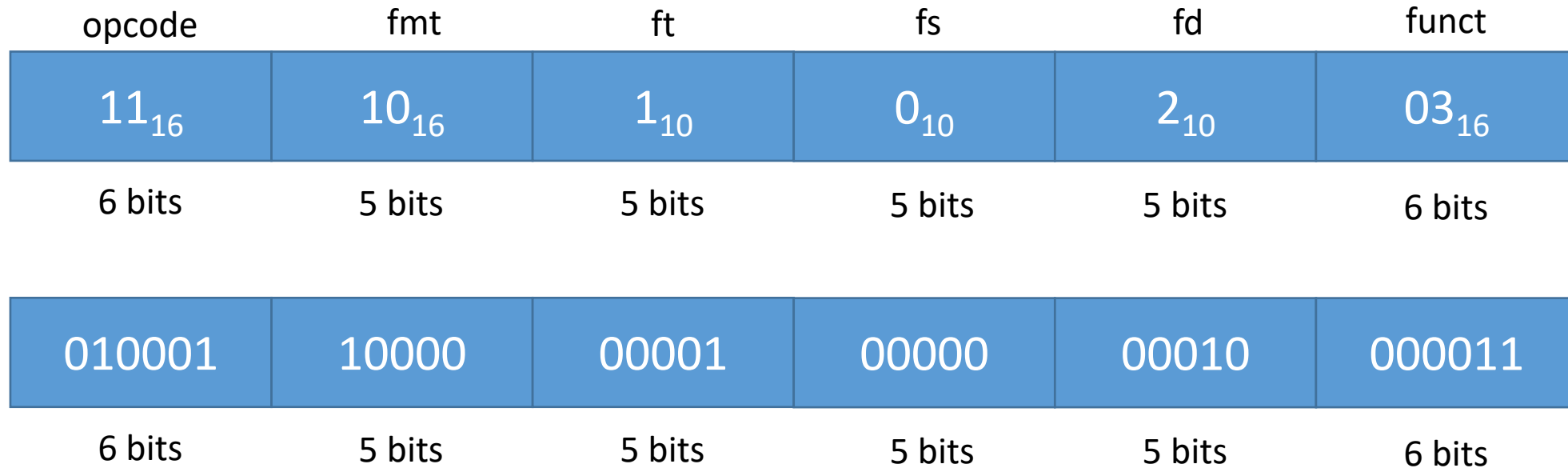
- Assembly Language: **sub.s \$f2, \$f0, \$f1**
- Machine Language: **0100 0110 0000 0001 0000 0000 1000 0001**

Floating Point Register Format Instructions



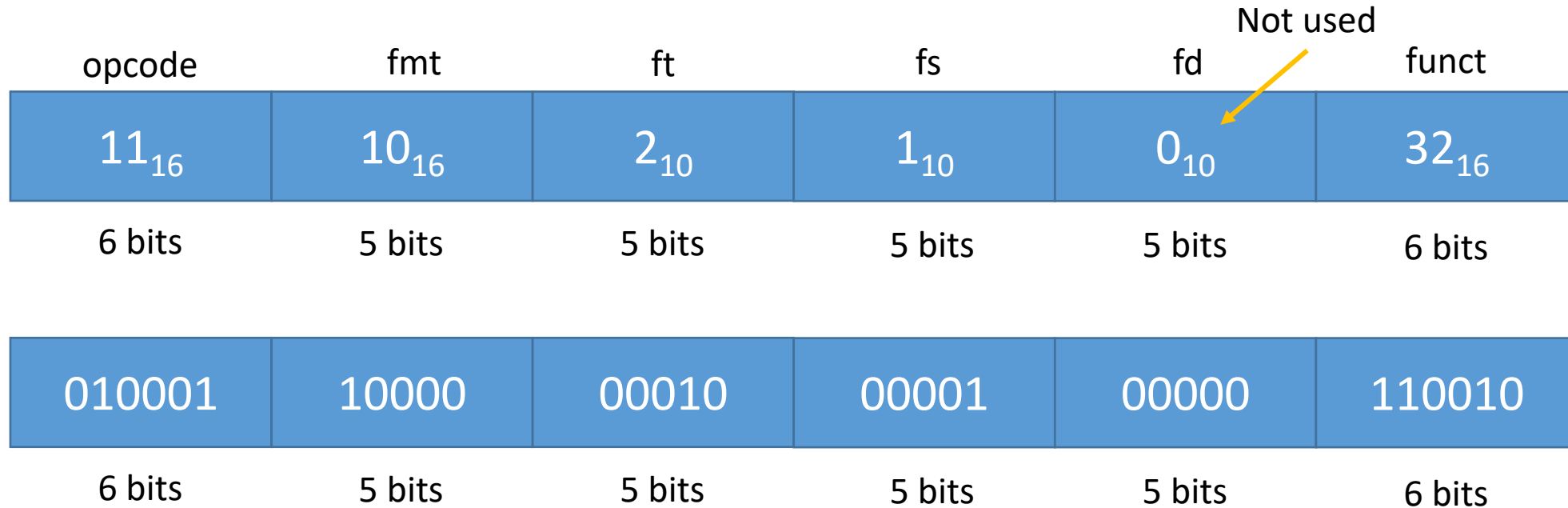
- Assembly Language: **mul.s \$f2, \$f0, \$f1**
- Machine Language: **0100 0110 0000 0001 0000 0000 1000 0010**

Floating Point Register Format Instructions



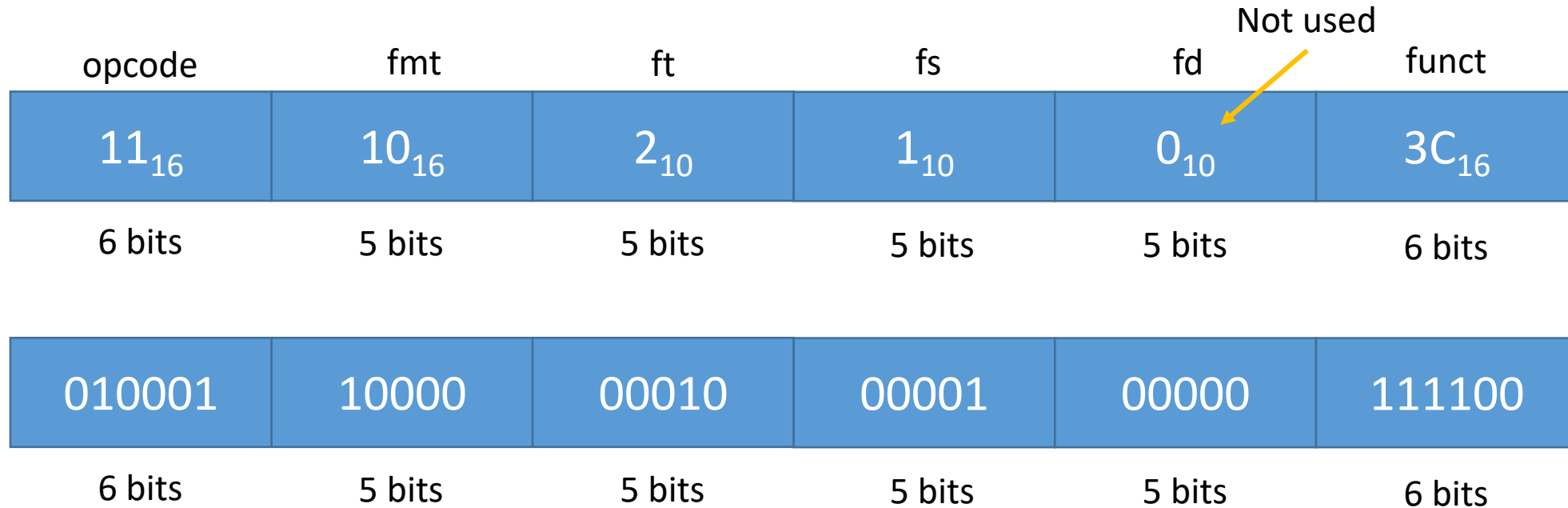
- Assembly Language: **div.s \$f2, \$f0, \$f1**
- Machine Language: **0100 0110 0000 0001 0000 0000 1000 0011**

Floating Point Register Format Instructions



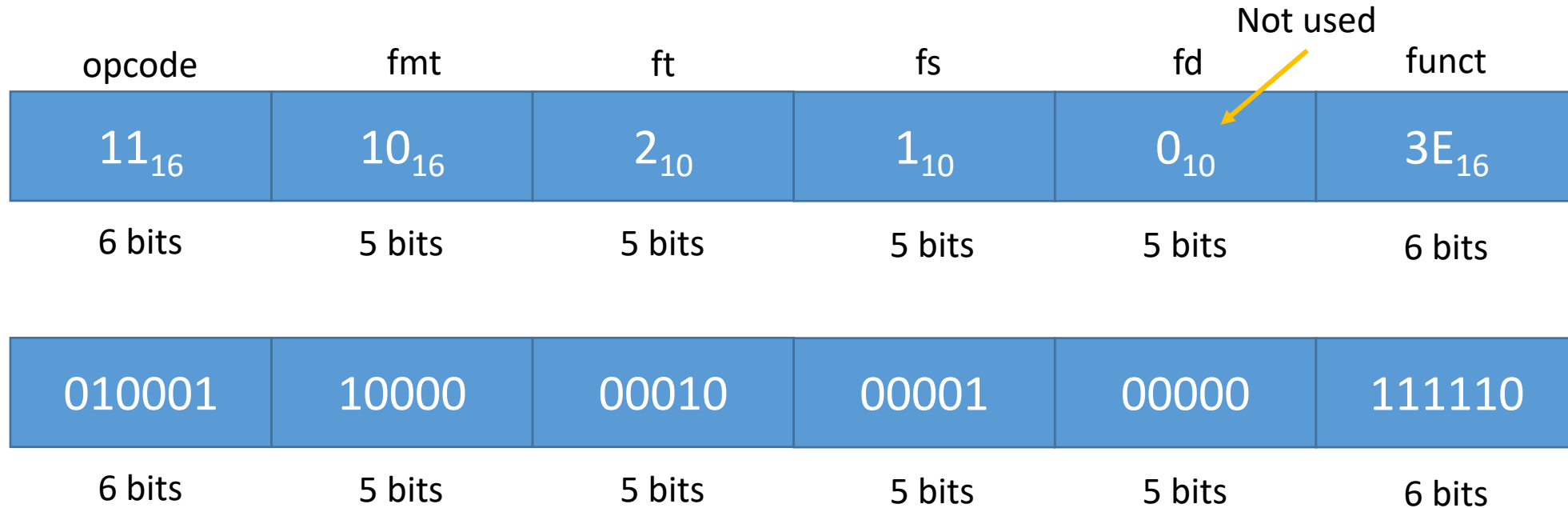
- Assembly Language: **c.eq.s \$f1, \$f2**
- Machine Language: **0100 0110 0000 0010 0000 1000 0011 0010**

Floating Point Register Format Instructions



- Assembly Language: **c.lt.s \$f1, \$f2**
- Machine Language: **0100 0110 0000 0010 0000 1000 0011 1100**

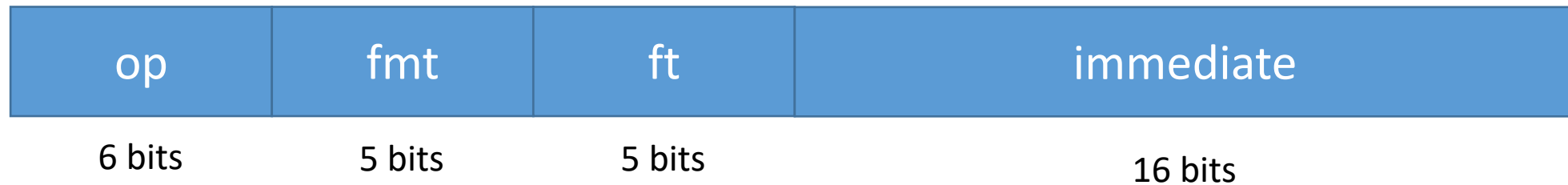
Floating Point Register Format Instructions



- Assembly Language: **c.le.s \$f1, \$f2**
- Machine Language: **0100 0110 0000 0010 0000 1000 0011 1110**

Floating Point Immediate Format Instructions

- MIPS Floating Point Immediate Format Instruction

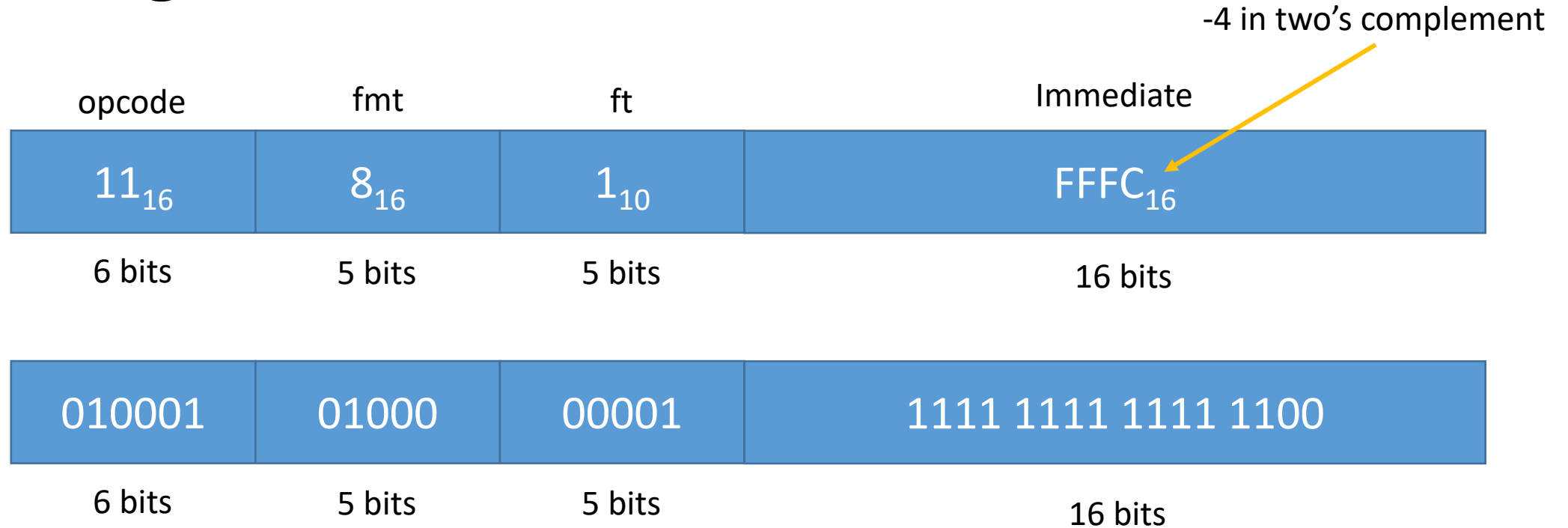


- op – Opcode
- fmt – Format
- ft – Source register
- immediate – Second/Destination register

Floating Point Immediate Format Instructions

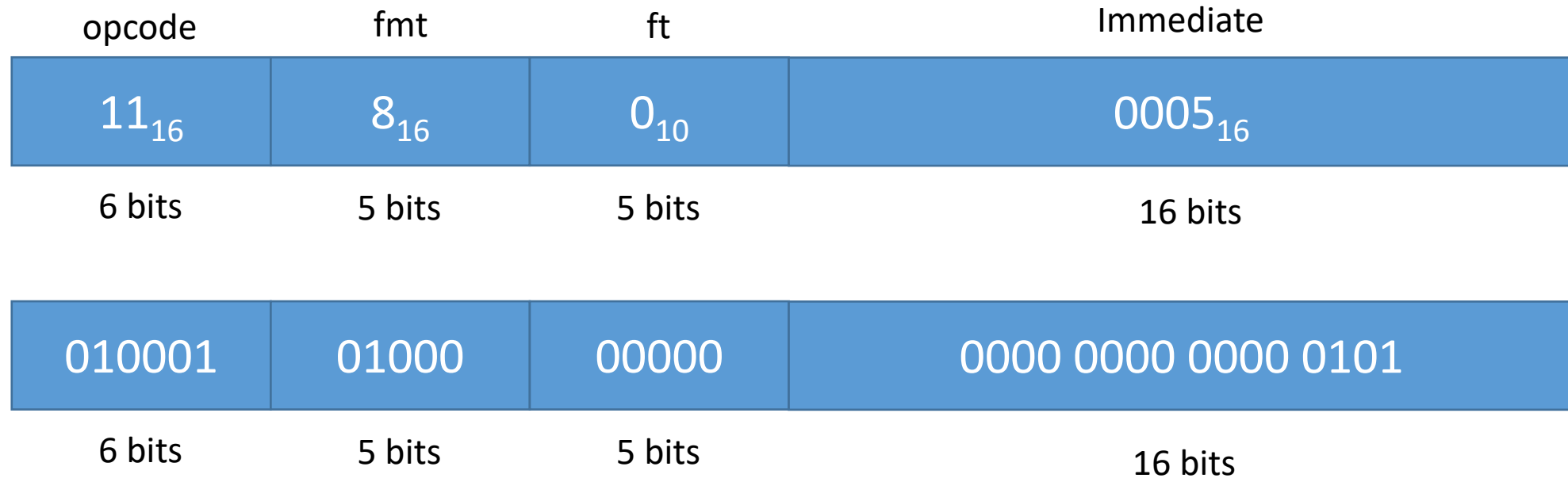
- We previously saw the two branching instructions for comparisons of floating point numbers
 - **bc1f** and **bc1t**
 - Relies on the value of the condition code, set by a conditional instruction like **c.le.s** or **c.eq.d**
- Branching to a label works in a similar fashion to the previous examples of advancing or backing up X number of instructions.

Floating Point Immediate Format Instructions



- Assembly Language: **bc1t** *Label/offset*
- Machine Language: **0100 0101 0000 0001 1111 1111 1111 1100**

Floating Point Immediate Format Instructions

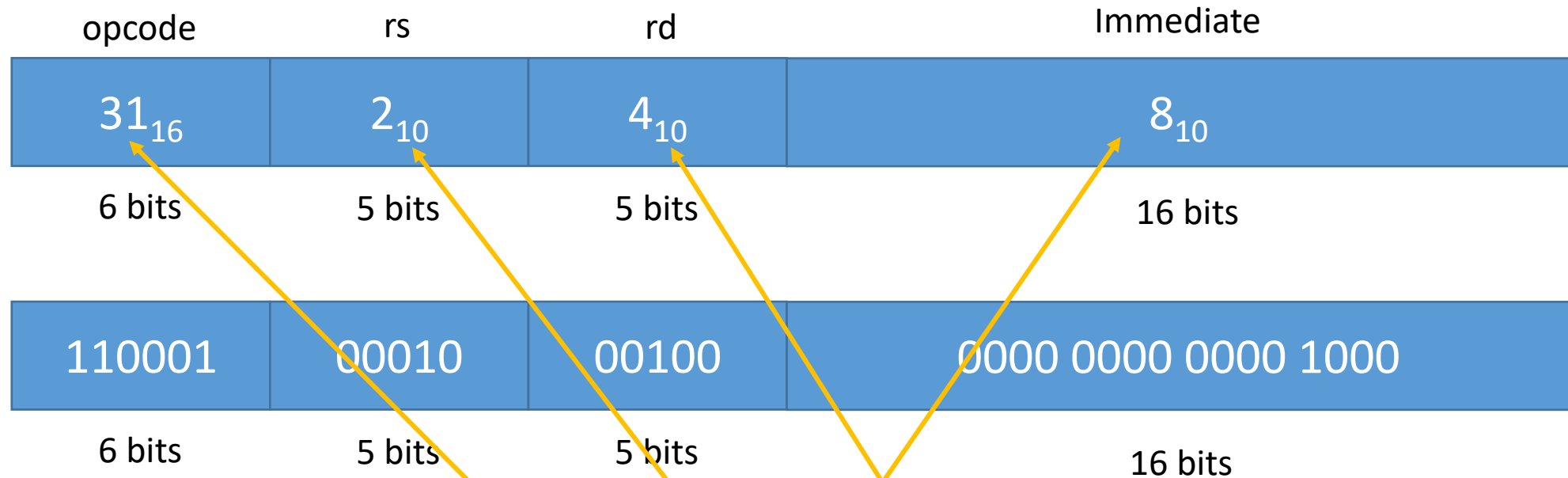


- Assembly Language: **bc1f** *Label/offset*
- Machine Language: **0100 0101 0000 0000 0000 0000 0000 0101**

Floating Point Immediate Format Instructions

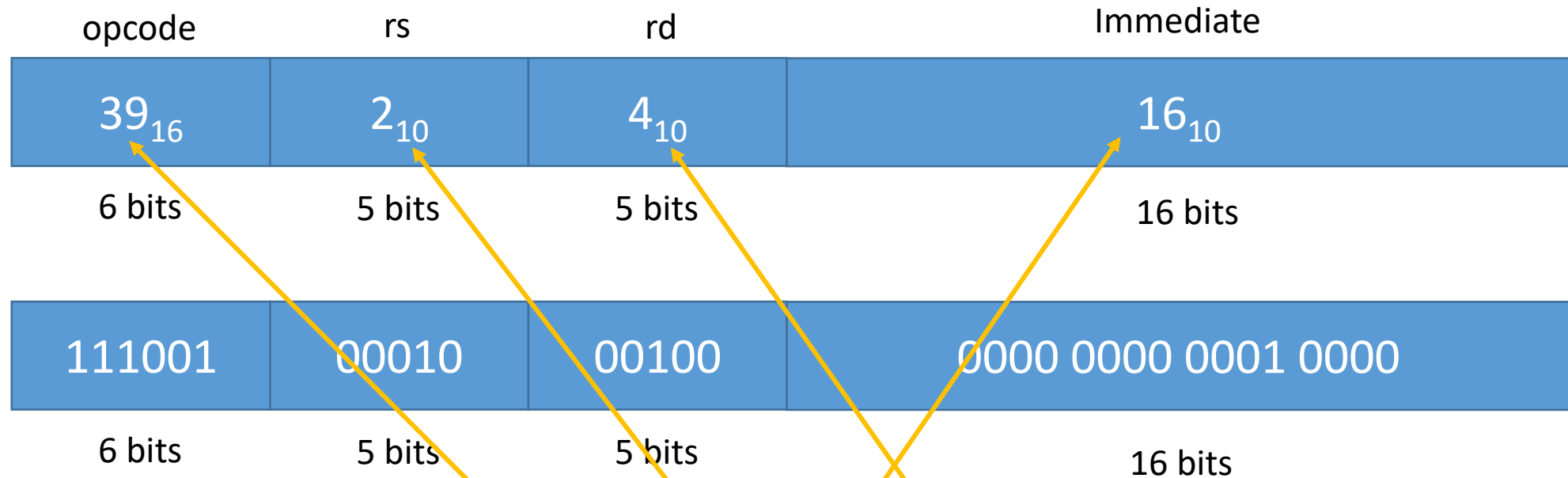
- Memory reference instructions for floating points use I Format instructions, *not* FI Format.
 - **swc1** and **lwc1**
- The machine language will be practically identical to the **lw** and **sw** instructions

Floating Point Immediate Format Instructions



- Assembly Language: **lwc1 \$f2, 8(\$a0)**
- Machine Language: **1100 0100 0100 0100 0000 0000 0000 1000**

Floating Point Immediate Format Instructions



- Assembly Language: **swc1 \$f2, 16(\$a0)**
- Machine Language: **1110 0100 0100 0100 0000 0000 0001 0000**